

 اوتنورسيتي مليسيا قهق السلطان عبدالله UNIVERSITI MALAYSIA PAHANG AL-SULTAN ABDULLAH	COURSE NAME: OBJECT-ORIENTED DESIGN & IMPLEMENTATION		COURSE CODE DRC1213	/30%
	ASSESSMENT: Group Project			
	LECTURER: Ts. Kirahman Bin Ab Razak			
	DATE: W13		TIME:	
	GROUP INFORMATION			
GROUP NAME				
SECTION	03	LAB SECTION	03A	

PROJECT TITLE:

Meeting Room Booking System

LECTURER NAME: Prof. Dato' Ts. Dr Kamal Zuhairi Bin Zamli			
TEAM MEMBERS :			
NO	MATRIC NO	NAME	SECTION
1	RC25071	ADRIEL CHONG SHEN KHAI	01
2	RC25293	MUHAMMAD ATIQULLAH BIN MUHD AKMAL	01
3	RC25314	MUHAMMAD ARISH AKMAR BIN AIDIL AKMAR	01
4.	RC25250	WAN MUHAMMAD ADAM BIN WAN FAIKAL SHAH	01

Table of Contents

Chapter 1: System Introduction & Vision

Chapter 2: Functional Requirements

Chapter 3: Non-Functional Requirements

Chapter 4: OOP Design (Use Case, Sequence, Class Diagrams)

Chapter 5: Class Relationships

Chapter 6: Encapsulation

Chapter 7: Inheritance

Chapter 8: Polymorphism

Chapter 9: Abstraction, Interface & Array Storage

Chapter 1: System Introduction & Vision

Introduction:

The Meeting Room Booking System is a Java-based application developed to streamline the reservation process of shared spaces within an organization or campus. It replaces manual scheduling methods, reducing the risk of overlapping bookings and enhancing overall resource management efficiency

Objective:

The objective of this system is to provide a reliable and user-friendly platform where users can check real-time room availability and make reservations. Additionally, it enables administrators to manage room resources effectively by applying OOP principles including Encapsulation, Inheritance, Polymorphism, and Abstraction.

Chapter 2: Functional Requirements (FR)

These describe what the system must do.

ID	Functional Requirement
FR-01	The system shall validate room availability by iterating through the Room array
FR-02	The system shall restrict any booking duration to a maximum of 4 hours.
FR-03	The system shall instantiate a new Booking object upon successful selection
FR-04	The system shall update the isAvailable status of the selected Room to false

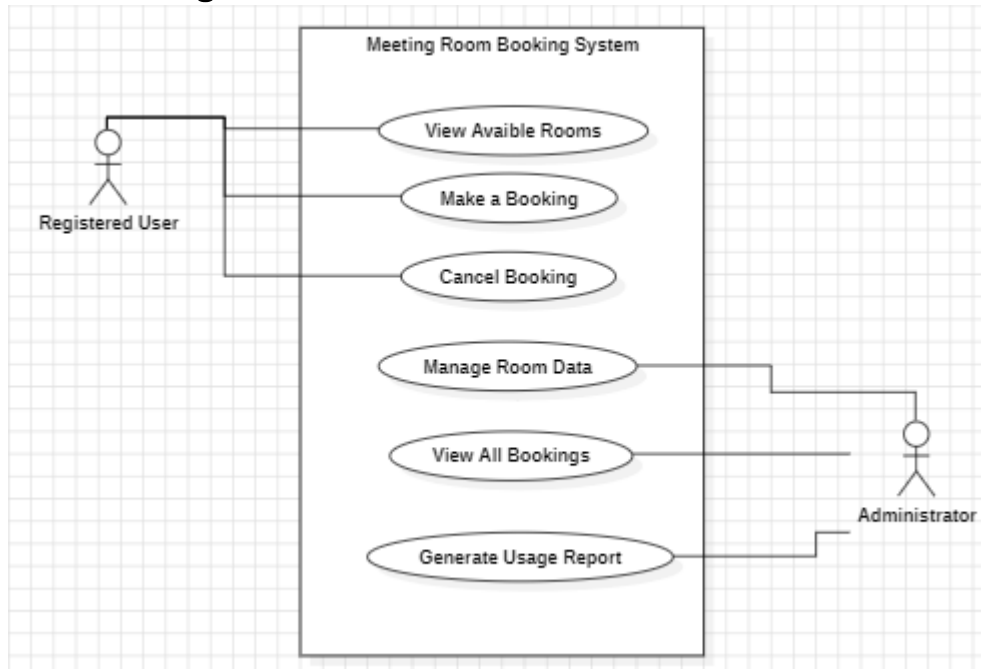
Chapter 3: Non-Functional Requirements (NFR)

These describe how the system should perform and its constraints

ID	Functional Requirement
NFR-01	The System shall prevent concurrent bookings of the same room for overlapping time periods.
NFR-02	The System shall respond to room availability queries within a maximum of 2 seconds
NFR-03	The System shall provide a user-friendly interface that adheres to standard UI/UX design principles.

Chapter 4: OOP Design

4.1 Use Case Diagram

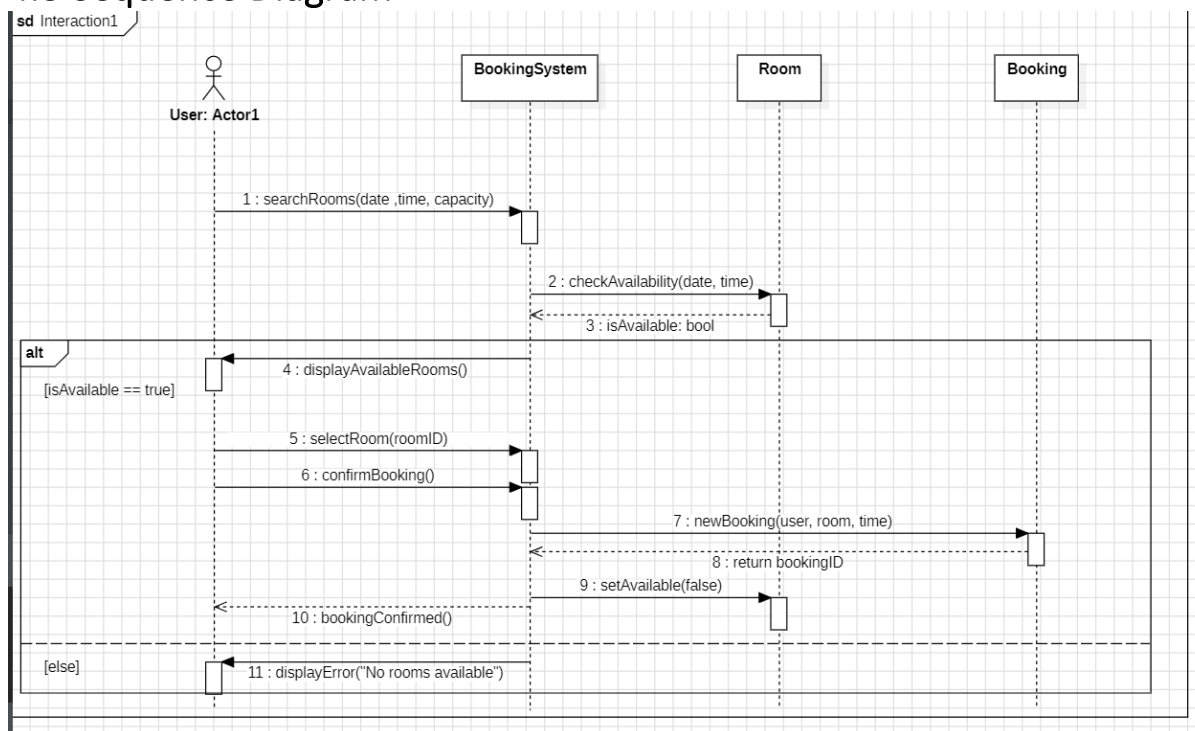


4.2 Use Case Description

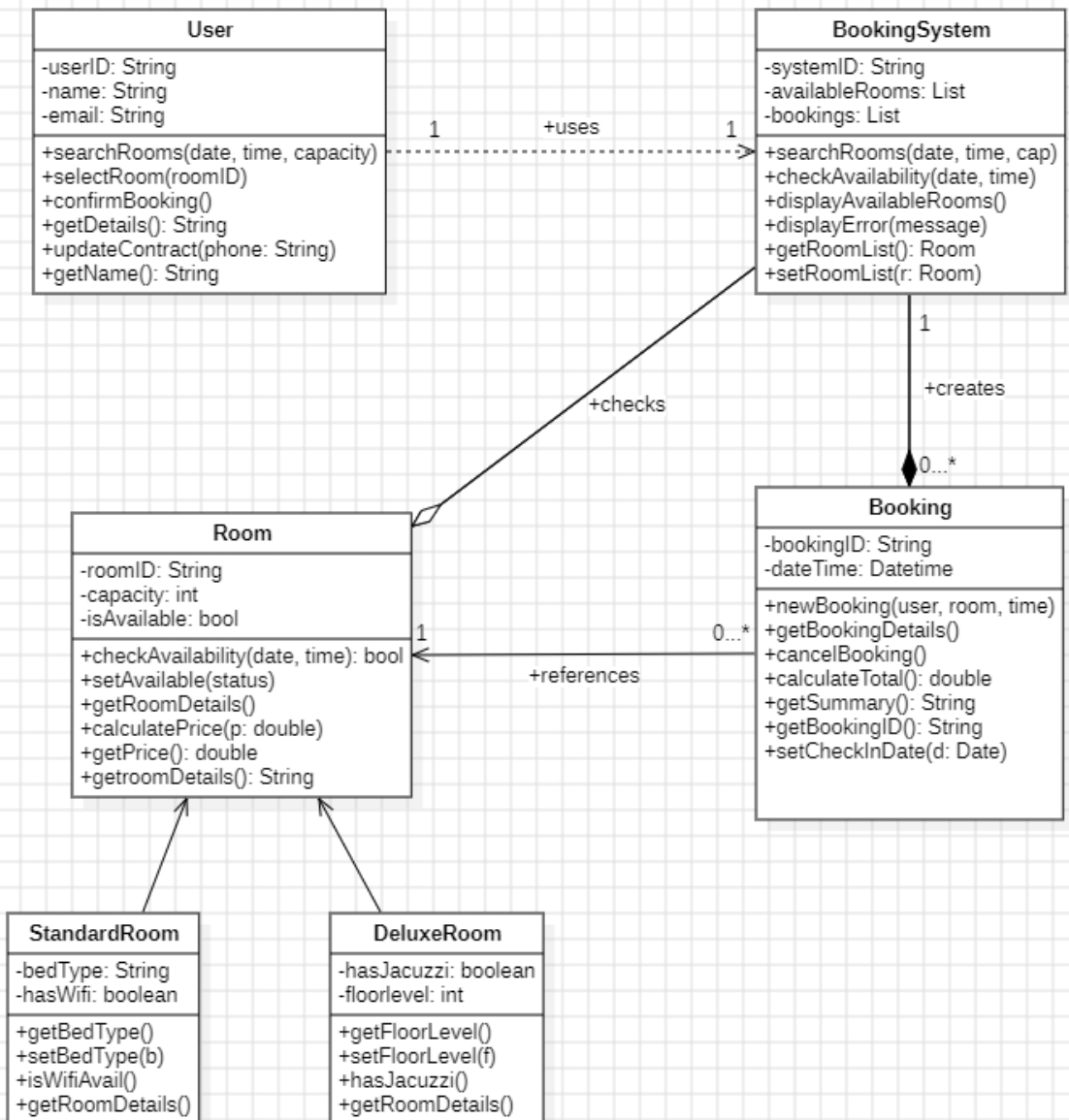
Section	Details
Use Case Name	Book Meeting Room
Brief Description	This use case allows a Registered User to reserve a specific meeting room by providing time slots and capacity requirements.
Actor	Registered User
Pre-condition	The user is logged into the application. The system displays the main function menu for the CLI or GUI.
Basic Flow	1. The use case starts when the user selects the “Book Room” option. 2. The system prompts the user to input the desired date, start time and end time. 3. The user enters the booking details 4. The system validates available rooms ([E1: No room available]) (details: The system iterates through its in-memory array of room object to find rooms where isAvailable == true for the requested time.) 5. The system displays a list of available rooms and their capacities.

	- The user selects their preferred room. - The system creates the user's booking info. (details: The system instantiates a new Booking object in memory. It links the active User object and the selected Room object to this new reservation. The system updates the room object status to isAvailable = false) - The system displays a booking summary. - The use case ends.
Alternate Flow	N/A
Exception Flow	[E1: No room available] - The system displays a "Room Fully booked" message. - The use case continues at Step 2 to allow the user to select a different time.
Post-Condition	The reservation is successfully stored in the array, and the system returns to the main menu.
Constraints	The duration of the booking cannot exceed 4 hours per session.

4.3 Sequence Diagram



4.4 Class Diagram



Chapter 5: Class Relationships

Class relationships define how objects interact and depend on each other. In this system, three types of relationships are implemented: Composition, Aggregation, and Association.

5.1 Composition – Booking contains User and Room

Composition is a strong 'has-a' relationship where the contained objects cannot exist independently. In this system, a Booking object is meaningless without a User and a Room — it owns references to both and links them at creation time.

This directly matches the Class Diagram from Phase 1, which shows Booking referencing both Room and User with a filled diamond (composition).

Code implementation (Booking.java):

```
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

public class Booking implements Bookable {

    // Encapsulated fields
    private String bookingID;
    private User user;
    private Room room;
    private String date;
    private String startTime;
    private String endTime;
    private double totalHours;
    private double totalPrice;
    private boolean isActive;
    private LocalDateTime checkInDate;

    private static int bookingCounter = 1;

    public Booking(User user, Room room, String date, String startTime, String endTime) {
        this.bookingID = String.format("BK%03d", bookingCounter++);
        this.user = user;
        this.room = room;
        this.date = date;
        this.startTime = startTime;
        this.endTime = endTime;
        this.totalHours = calculateDuration(startTime, endTime);
        this.totalPrice = room.calculatePrice(totalHours);
        this.isActive = true;
        this.checkInDate = LocalDateTime.now();

        room.setAvailable(false);
    }
}
```

5.2 Aggregation – BookingSystem holds Room array

Aggregation is a weaker 'has-a' relationship where the contained objects can exist independently. The BookingSystem holds an array of Room objects, but rooms can exist without the system.

Code implementation (BookingSystem.java):

```
public class BookingSystem {  
  
    private String systemID;  
  
    private Room[] availableRooms;  
    private Booking[] bookings;  
    private int roomCount;  
    private int bookingCount;  
  
    private static final int MAX_ROOMS = 20;  
    private static final int MAX_BOOKINGS = 100;  
    private static final int MAX_BOOKING_HOURS = 4;
```

5.3 Association – User uses BookingSystem

Association is a 'uses-a' relationship where one object uses another but neither owns the other. In this system, the User interacts with BookingSystem to search and book rooms, but both can exist independently.

This matches the Class Diagram from Phase 1, which shows a dashed arrow from User to BookingSystem labeled +uses.

Code implementation (Main.java):

```
public class Main {  
  
    static Scanner sc = new Scanner(System.in);  
    static BookingSystem system = new BookingSystem("SYS-001");  
    static User currentUser = null;  
  
    Booking booking = system.createBooking(currentUser, selectedRoom, date, startTime, endTime);  
    if (booking == null) return;
```


Chapter 6: Encapsulation

Encapsulation is the OOP principle of hiding internal data and only exposing it through controlled methods (getters and setters). All classes in this system declare their fields as private and provide public accessor methods.

6.1 Encapsulation in Room.java

The Room class encapsulates all room attributes. Direct access to fields is prevented — all interaction happens through methods.

```
public abstract class Room {  
  
    private String roomId;  
    private int capacity;  
    private boolean isAvailable;  
    ✦ private double pricePerHour;-  
  
    // Constructor  
    public Room(String roomId, int capacity, double pricePerHour) {  
        this.roomID = roomId;  
        this.capacity = capacity;  
        this.isAvailable = true;  
        this.pricePerHour = pricePerHour;  
    }  
  
    // --- Getters & Setters ---  
    public String getRoomID() { return roomId; }  
    public int getCapacity() { return capacity; }  
    public boolean isAvailable() { return isAvailable; }  
    public double getPricePerHour() { return pricePerHour; }  
  
    public void setAvailable(boolean status) { this.isAvailable = status; }  
    public void setPricePerHour(double p) { this.pricePerHour = p; }
```

6.2 Encapsulation in User.java

The User class similarly protects user data. The phone number can only be updated via the `updateContact()` method, not by direct field access.

```
public class User {  
    // Encapsulated fields  
    private String userID;  
    private String name;  
    private String email;  
    private String phone;  
  
    public User(String userID, String name, String email) {  
        this.userID = userID;  
        this.name = name;  
        this.email = email;  
        this.phone = "";  
    }  
  
    // Getters & Setters (Encapsulation)  
    public String getUserID() { return userID; }  
    public String getName() { return name; }  
    public String getEmail() { return email; }  
    public String getPhone() { return phone; }  
  
    public void updateContact(String phone) {  
        this.phone = phone;  
    }  
  
    public String getDetails() {  
        return String.format("User ID: %s | Name: %s | Email: %s | Phone: %s",  
            userID, name, email, phone.isEmpty() ? "N/A" : phone);  
    }  
}
```

Chapter 7: Inheritance

Inheritance allows a subclass to reuse and extend the behaviour of a parent class. In this system, StandardRoom and DeluxeRoom both extend the abstract Room class, inheriting its common attributes and methods while adding their own specific features.

7.1 Class Hierarchy

Room (abstract parent) <- StandardRoom and DeluxeRoom (concrete subclasses)

Both subclasses inherit from Room: roomID, capacity, isAvailable, pricePerHour, calculatePrice(), checkAvailability(), and all getters/setters.

7.2 StandardRoom extends Room

StandardRoom adds bedType and hasWifi attributes specific to standard meeting rooms.

```
public class StandardRoom extends Room {  
  
    private String bedType;  
    private boolean hasWifi;  
  
    public StandardRoom(String roomID, int capacity, double pricePerHour,  
                        String bedType, boolean hasWifi) {  
        super(roomID, capacity, pricePerHour);  
        this.bedType = bedType;  
        this.hasWifi = hasWifi;  
    }  
  
    // Getters & Setters  
    public String getBedType() { return bedType; }  
    public void setBedType(String b) { this.bedType = b; }  
    public boolean isWifiAvail() { return hasWifi; }  
}
```

7.3 DeluxeRoom extends Room

DeluxeRoom adds hasJacuzzi and floorLevel attributes specific to deluxe rooms.

```
public class DeluxeRoom extends Room {  
  
    private boolean hasJacuzzi;  
    private int floorLevel;  
  
    public DeluxeRoom(String roomID, int capacity, double pricePerHour,  
                       boolean hasJacuzzi, int floorLevel) {  
        super(roomID, capacity, pricePerHour); // Call parent constructor  
        this.hasJacuzzi = hasJacuzzi;  
        this.floorLevel = floorLevel;  
    }  
  
    // Getters & Setters  
    public boolean hasJacuzzi() { return hasJacuzzi; }  
    public int getFloorLevel() { return floorLevel; }  
    public void setFloorLevel(int f) { this.floorLevel = f; }
```

Chapter 8: Polymorphism

Polymorphism allows objects of different types to be treated as objects of a common parent type. At runtime, Java determines which overridden method to call based on the actual object type.

8.1 Method Overriding – getRoomDetails()

The abstract method `getRoomDetails()` is declared in `Room` and overridden differently in each subclass. When called on a `Room` reference, Java automatically uses the correct version based on the actual object type.

StandardRoom override:

```
// Polymorphism: overrides abstract method from Room
@Override
public String getRoomDetails() {
    return String.format(
        "[ STANDARD ROOM ]\n" +
        "  Room ID      : %s\n" +
        "  Capacity     : %d persons\n" +
        "  Bed Type     : %s\n" +
        "  Wifi         : %s\n" +
        "  Price/hr     : RM%.2f\n" +
        "  Available    : %s",
        getRoomID(), getCapacity(), bedType,
        hasWifi ? "Yes" : "No",
        getPricePerHour(),
        isAvailable() ? "Yes" : "No"
    );
}
```

DeluxeRoom override:

```
// Polymorphism: overrides abstract method from Room
@Override
public String getRoomDetails() {
    return String.format(
        "[ DELUXE ROOM ]\n" +
        "  Room ID      : %s\n" +
        "  Capacity     : %d persons\n" +
        "  Floor Level  : %d\n" +
        "  Jacuzzi      : %s\n" +
        "  Price/hr     : RM%.2f\n" +
        "  Available    : %s",
        getRoomID(), getCapacity(), floorLevel,
        hasJacuzzi ? "Yes" : "No",
        getPricePerHour(),
        isAvailable() ? "Yes" : "No"
    );
}
```

8.2 Polymorphism in Action – displayAvailableRooms()

BookingSystem iterates through its Room array and calls methods on each element. Because Room is abstract, the runtime type (StandardRoom or DeluxeRoom) determines which method executes — this is runtime polymorphism.

```
// Polymorphism in action: iterates Room array, calls getRoomDetails() on each
// (actual runtime type may be StandardRoom or DeluxeRoom)
public void displayAvailableRooms() {
    System.out.println("\n ===== AVAILABLE ROOMS =====");
    boolean found = false;
    for (int i = 0; i < roomCount; i++) {
        if (availableRooms[i].isAvailable()) {
            System.out.println(availableRooms[i]); // calls overridden toString
            found = true;
        }
    }
    if (!found) System.out.println(" No rooms currently available.");
    System.out.println(" =====");
}
```

Chapter 9: Abstraction, Interface & Array Storage

9.1 Abstraction – Abstract Class Room

Abstraction hides implementation details and only exposes what is necessary. The Room class is declared abstract — it cannot be instantiated directly. It forces all subclasses to implement the getRoomDetails() method, ensuring every room type provides its own description.

```

public abstract class Room {

    private String roomID;
    private int capacity;
    private boolean isAvailable;
    private double pricePerHour;

    // Constructor
    public Room(String roomID, int capacity, double pricePerHour) {
        this.roomID = roomID;
        this.capacity = capacity;
        this.isAvailable = true;
        this.pricePerHour = pricePerHour;
    }

    // --- Getters & Setters ---
    public String getRoomID() { return roomID; }
    public int getCapacity() { return capacity; }
    public boolean isAvailable() { return isAvailable; }
    public double getPricePerHour() { return pricePerHour; }

    public void setAvailable(boolean status) { this.isAvailable = status; }
    public void setPricePerHour(double p) { this.pricePerHour = p; }

    // Check availability for a given time duration
    public boolean checkAvailability(String date, String time) {
        return isAvailable;
    }

    // Calculate total price for given hours
    public double calculatePrice(double hours) {
        return pricePerHour * hours;
    }

    // Abstract method – must be implemented by subclasses (Abstraction)
    public abstract String getRoomDetails();

    @Override

```

9.2 Interface – Bookable

An interface defines a contract that implementing classes must fulfil. The Bookable interface declares three methods that any bookable entity must implement. The Booking class implements this interface, guaranteeing it provides booking, cancellation, and summary functionality.

```
public interface Bookable {
    boolean makeBooking(String date, String startTime, String endTime);
    boolean cancelBooking(String bookingID);
    String getBookingSummary();
}

// --- Bookable Interface implementations ---

@Override
public boolean makeBooking(String date, String startTime, String endTime) {
    return isActive;
}

@Override
public boolean cancelBooking(String bookingID) {
    if (this.bookingID.equals(bookingID) && isActive) {
        isActive = false;
        room.setAvailable(true); // Free up the room
        return true;
    }
    return false;
}
```


9.3 Array as Temporary Storage (CO3 Requirement)

As required by the CO3 assessment criteria, the system uses Java arrays (not ArrayList or other collections) as temporary in-memory storage for rooms and bookings. Data is stored only while the program is running.

```
public class BookingSystem {  
  
    private String systemID;  
  
    private Room[] availableRooms;  
    private Booking[] bookings;  
    private int roomCount;  
    private int bookingCount;  
  
    public void addRoom(Room room) {  
        if (roomCount < MAX_ROOMS) {  
            availableRooms[roomCount++] = room;  
        } else {  
            System.out.println("  [!] Room list is full.");  
        }  
    }  
  
    public Room[] getRoomList() {  
        Room[] result = new Room[roomCount];  
        for (int i = 0; i < roomCount; i++) result[i] = availableRooms[i];  
        return result;  
    }  
}
```

```
public Room[] searchRooms(String date, String time, int requiredCapacity) {
    Room[] results = new Room[roomCount];
    int count = 0;
    for (int i = 0; i < roomCount; i++) {
        Room r = availableRooms[i];
        if (r.checkAvailability(date, time) && r.getCapacity() >= requiredCapacity) {
            results[count++] = r;
        }
    }
    Room[] trimmed = new Room[count];
    for (int i = 0; i < count; i++) trimmed[i] = results[i];
    return trimmed;
}

public boolean checkAvailability(String date, String time) {
    for (int i = 0; i < roomCount; i++) {
        if (availableRooms[i].checkAvailability(date, time)) return true;
    }
    return false;
}

public Room findRoomByID(String roomID) {
    for (int i = 0; i < roomCount; i++) {
        if (availableRooms[i].getRoomID().equalsIgnoreCase(roomID)) {
            return availableRooms[i];
        }
    }
    return null;
}
```